

Notes:

- HW4 due Nov 6
- HW5 posted

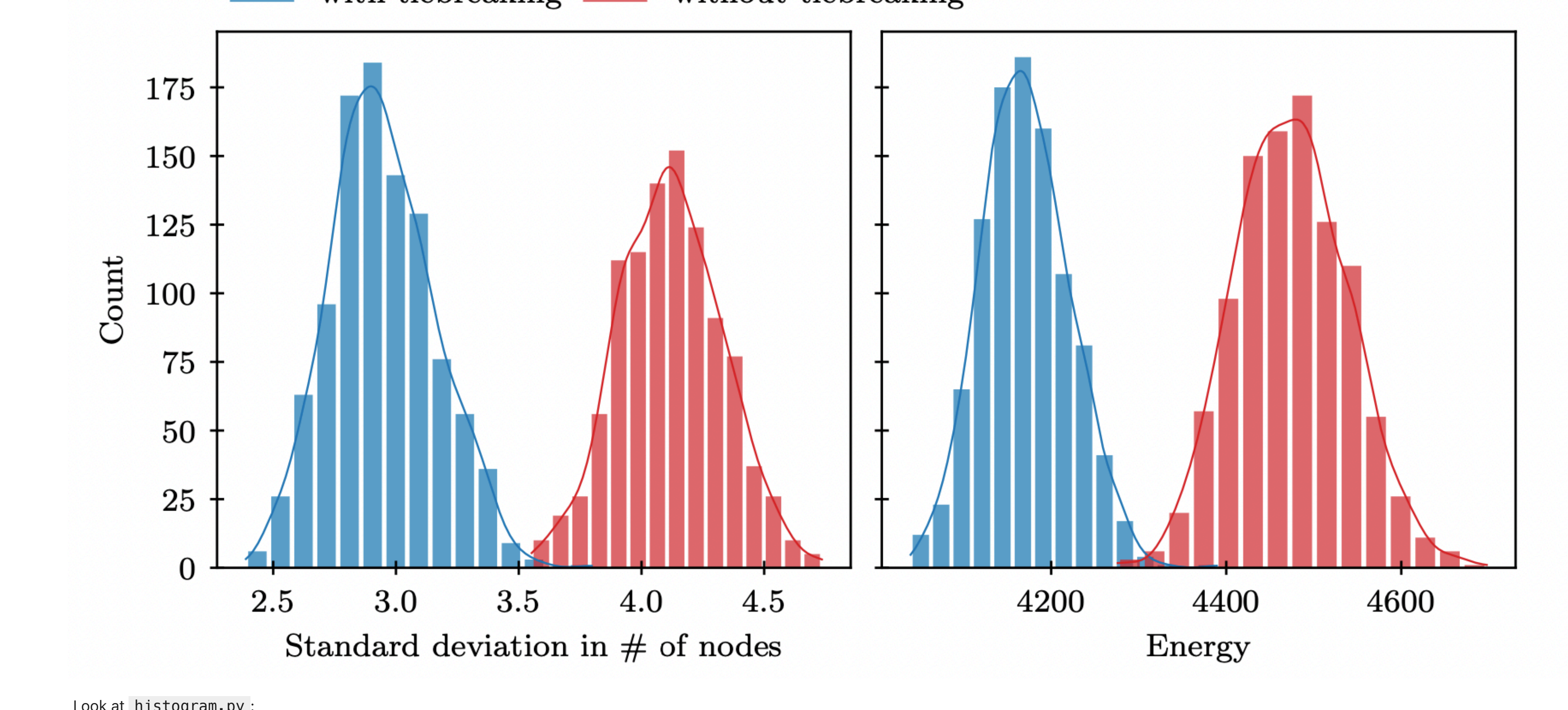
- Last class:
- plotting and setting matplotlib defaults
 - plotting from pandas
 - purposes of a graphic
 - aspects of a plot
 - data, variable transformations, data frame, data scale, coordinates, graph, guides
 - reduced colour vision and plotting
 - started plotting examples

In [2]:

```
import matplotlib.pyplot as plt
import matplotlib as mpl
my_plot_rc = {'font.size': 18,
              'lines.linewidth': 2,
              'savefig.bbox': 'tight'}
mpl.rcParams.update(my_plot_rc)
```

In [1]:

```
from IPython import display
display.Image("histogram.png")
```



Look at `histogram.py`:

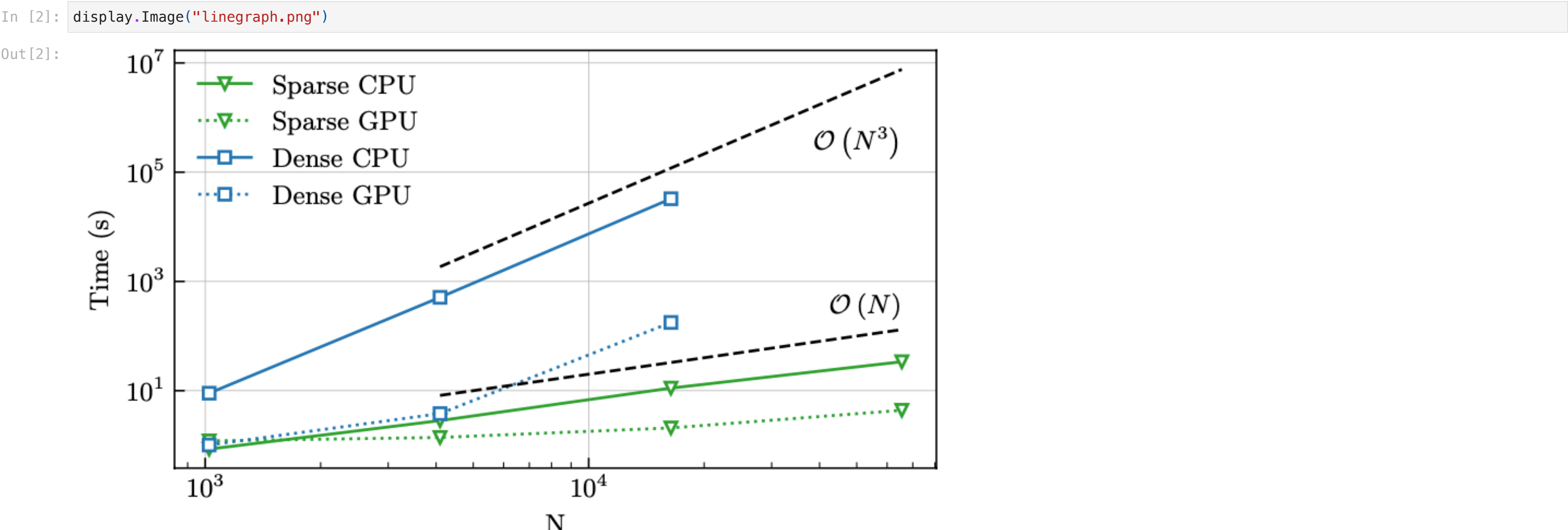
VERY IMPORTANT: this plots data saved (in numpy's .npz format) from another script

- **ALWAYS** separate your data generation from your plotting scripts. Data generation is usually expensive (running a code, testing algorithms, "crunching numbers"), while visualization is cheap. Common to tweak figure design many times. Don't want to have to wait for data to generate again when all you want to do is tweak an axis label or change a line style...

In `histogram.py`:

- update MPL defaults and create a dictionary of common kwargs for all plots (style of histogram and kde lines)
- plots, selecting which data to plot from dataframe, which axes from subplots, passing common kwargs so all plots look the same
- legend and labels, note legend goes outside the plot
- command to save graphic as pdf file

In contrast to these examples (bar graphs and histograms, discrete counts of data), much of our data is often in the form of time series or similar, with clear ordering. Consider



- plotting computing time for certain operation on data of size N, on CPU and GPU, for sparse and dense matrix implementations.
- Again, look at `linegraph.py`:
- data stored from some external source (few experiments, so just tabulated directly)
 - set some defaults
 - use `plt.figure()` then `ax = plt.gca()` to use subplot (axes) style commands that we have been using (just for consistency)
 - 3 sets of plots:
 - Sparse timings, only differ in CPU/GPU linestyle
 - Dense timings, same as sparse, but for colour and marker
 - labelled scaling lines
 - axis labels, etc.

Think about components of a graphic:

- Data: timings from 4 numerical experiments
- Variable transformations: none, natural data is already ordered
- Data Frame: 2D line graph, natural "x vs. f(x)" structure
- Data Scale: N values extend over range from 1K to 65K, timings from 1 to 32K seconds.

Moreover, a general model for computational cost is the idea of big-O notation, where costs are modelled by

$$\text{cost} = c \cdot N^p$$

where c is some constant and p indicates how much work is required as a function of data size, N. With this model, we have

$$\log(\text{cost}) = p \cdot \log(N) + \log(c)$$

polynomial cost leads to straight line data in loglog scale

Contrast this with models of exponential growth

$$\text{cost} = c \cdot p^N$$

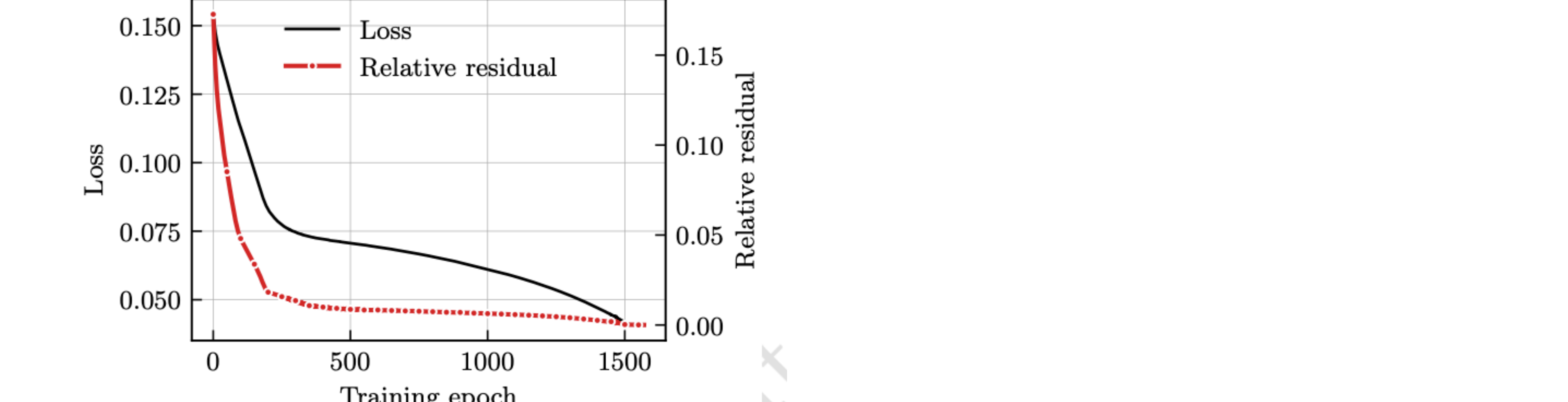
Now take logs on both sides, get

$$\log(\text{cost}) = N \cdot (\log p) + \log(c)$$

This gives straight lines on semilog plot (N vs. log(cost))

- Coordinates: natural N vs cost on log scale
- Graph: Fix colour and marker for sparse vs. dense, fix solid line for CPU and dotted line for GPU
- Guides: legend, axis labels, also have scaling lines

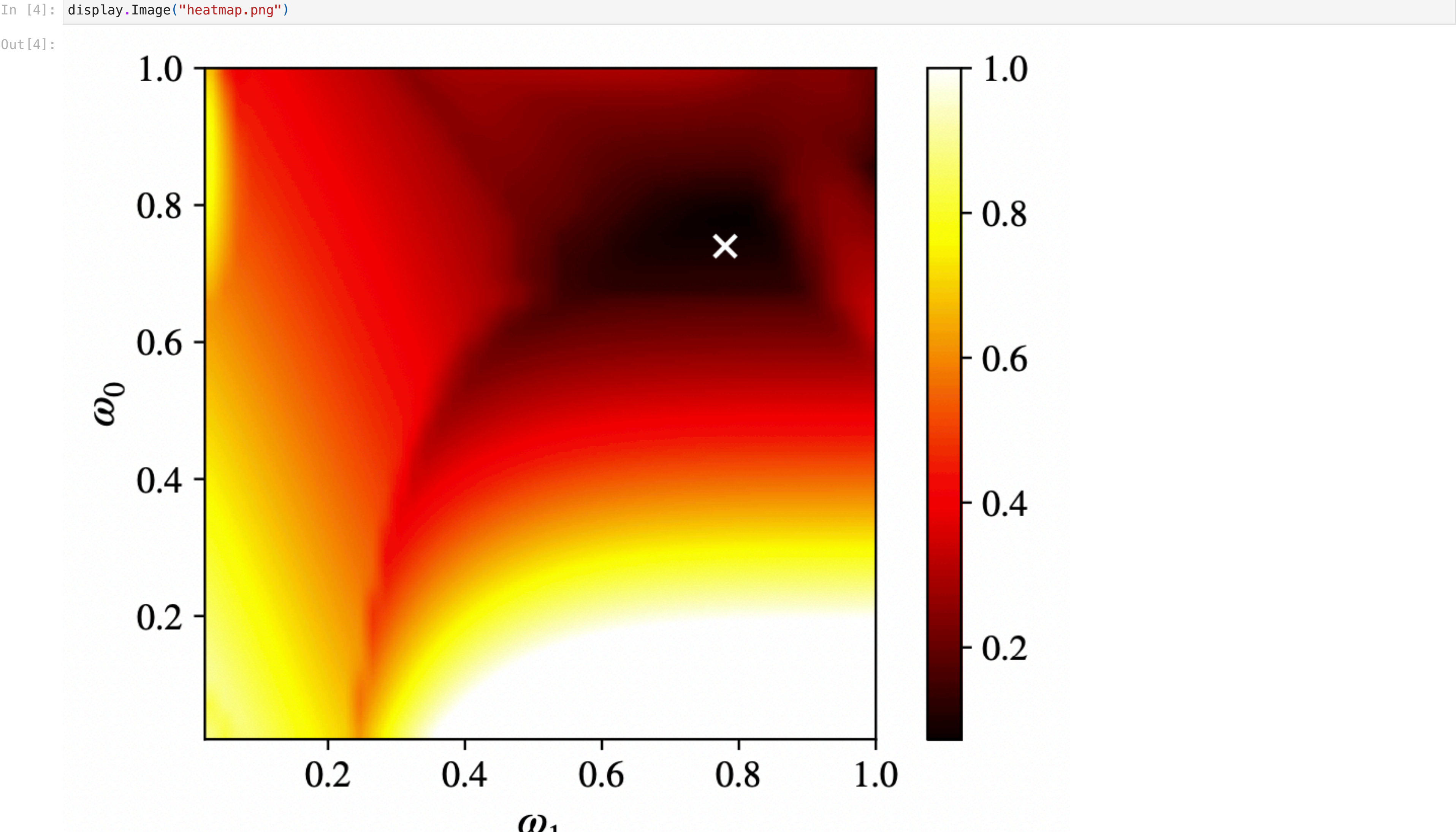
Sometimes we want to plot data with 2 different natural scales on same plot



Displays both training loss of a neural network and effectiveness of the learned algorithm (small relative residual implies a good algorithm) over a number of epochs

- Look at `dual_linegraph.py`:
- now data is stored in pickle format, storing a dictionary with data in pytorch tensors
 - plot for training loss (left-hand axis) looks normal, using `plt.subplots()` to get axis to use those commands
 - `ax.twinx()` creates a second y-axis that we can plot on.
 - No other real changes
- Can assess aspects as before: might question data scale for residual, since this goes to zero. Easy to try different scalings!
- See some noise in data, but also much more detail as we approach the end of training. Small drop in loss, but large drop in residual measure over last 100 iterations.

Some of our data more naturally fits as a "heat map" or image



This shows measured error reduction per step for an algorithm as a function of two algorithmic parameters, ω_0 and ω_1 , with optimal parameter marked.

- Look at `heatmap.py`:
- load data from file
 - This code has a bit more data stored within it (some analysis to record best parameters), could have stored all of this in one file.
 - use `imshow` for heat map (revisit in a moment)
 - labels, use `plt.colorbar()` for colorbar scale
 - use simple scatter plot to add marker for optimum parameters
- The basic `imshow` command displays array-like data. Here, `rho` is a 50x50 `ndarray` from numpy
- `origin` controls coordinates, `lower` puts the origin in the lower-left, default is upper-left
 - `extent` defines the x and y limits of data frame (useful to set this to avoid "empty space" in plot)
 - `cmap` chooses a colormap (here, we use `hot`), which is scaled for values from `vmin` to `vmax`. By default, the colour map scales over all of the values in the data. Here, we cap `vmax` at 1, as values greater than 1 are too big to be of interest.
 - `interpolation` controls the pixelation of the data. Here, we use bilinear interpolation to get a smooth image, but could also use `nearest` to see pixels

Best choice here is sequential colour maps to use. Two major classes are known as "sequential" and "diverging".

We tend to use sequential color maps for data that ranges from "small" to "big" without any distinctive values in between. Common choices here are `viridis`, `plasma`, `cool`, `hot`.

Diverging colormaps are useful when data has a distinct middle value, such as function values that vary between -1 and +1, with zero having some meaning or reference. Common diverging colormaps are `spectral`, `coolwarm`, and `bwr`.

Looks strange with this data, but good for data that is very oscillatory

Back to writing code in Python

Start looking in more depth at functions: Know basic way to declare a function

In []:

```
def my_function(parameter1, parameter2, ...):
    code
    more_code
    even more_code
    return something
```

Consider a simple example:

In [7]:

```
def print_date(month, day, year):
    date_string = str(month) + '/' + str(day) + '/' + str(year)
    print(date_string)
```

In [8]:

```
print_date(11,4,2025)
```

We can also name the arguments when we call the function, and then specify them in any order

In [9]:

```
print_date(day=4, month=11, year = 2025)
```

In [10]:

```
print_date(4,11,2025)
```

Remember, functions have to return *something*

In [11]:

```
value = print_date(11,4,2025)
print('returned value is ', value)
```

11/4/2025
returned value is None

If a function reaches the end of its scope without a return, automatically returns `None`.

Recall that Python determines scope for functions, for/while loops, if statements, etc., by indentation

Also be conscious that you cannot call a function before you define it:

In [12]:

```
my_func(2,3)

def my_func(x,y):
    print(f'{x}+{y} = {x+y}')
    return x+y
```

```
Traceback (most recent call last)
Cell In[12], line 1
----> 1 my_func(2,3)
      2 def my_func(x,y):
      3     print(f'{x}+{y} = {x+y}')

NameError: name 'my_func' is not defined
```

Two things:

- Cannot call `my_func` before it is defined
- Since this returns a `NameError`, function is not defined yet

In [13]:

```
my_func(2,3)
```

```
Traceback (most recent call last)
Cell In[13], line 1
----> 1 my_func(2,3)

NameError: name 'my_func' is not defined
```

In [14]:

```
def my_func(x,y):
    print(f'{x}+{y} = {x+y}')
    return x+y
```

In [15]:

```
my_func(2,3)
```

```
2+3 = 5
```

Out[15]:

```
5
```